Huzaifa Sa'ad

Backend Developer | Python | Django | REST APIs
houzsaad@gmail.com | +234 906 601 6407 | Jigawa, Nigeria
linkedin.com/in/huzaifa-sa-ad-7814a1377 | github.com/Houzsaad | houzsaad-portfolio.vercel.app

SUMMARY

Backend developer specialising in Django REST Framework, PostgreSQL, JWT authentication and payment integrations (Paystack, DodoData, VTPass). Built two VTU airtime and data platforms for paying clients, with wallet logic designed to credit and refund each transaction once, and Student Resource Hub, which reached 100 users within two days of testing. Currently leading a back-end team of 5 on a student admission platform as an intern. Seeking Junior Backend Developer and Backend Engineer roles.

TECHNICAL SKILLS

Languages: Python, JavaScript, HTML5, CSS3

Backend: Django, Django REST Framework, REST API design, JWT authentication, role-based access control, hashed PIN and password handling, rate limiting, unit testing

Payments and Integrations: Paystack (Pay with Transfer, signed webhooks), DodoData, VTPass, idempotent crediting and refunds, Resend (email), Cloudinary (file storage)

Databases: PostgreSQL (Neon, Render), SQLite, ORM query optimisation, pg_dump backup and restore

Frontend: React (Vite), React Router, Context API, Bootstrap, PWA

Tools and Deployment: Git, GitHub, Postman, Swagger/OpenAPI (drf-spectacular), Render, Vercel, Gunicorn, WhiteNoise

EXPERIENCE AND PROJECTS

Enrollaa – Student Admission Guidance Platform

Internship · Sep 2026 – Present

Backend Team Lead (Intern)

Website

- **Problem.** Students choose universities and courses without clear information on admission requirements or whether they qualify. Enrollaa helps them explore universities, check requirements, assess their eligibility and track their admission progress.
- **Role.** Leads a back-end team of 5: reviews code, assigns tasks and designs the architecture.
  Stack: Python, Django, PostgreSQL, Render

VTU Platform – Airtime and Data Reseller

Paid client project

Backend and Frontend Developer

Live site | GitHub

- **Problem.** Customers fund a wallet and buy airtime and data, and the business cannot afford money that is lost, credited twice, or stuck when a provider replies unclearly.
- **Impact.** Tested end to end with real airtime and data purchases. Each payment is credited once and each failed purchase is refunded once, so customers are not double-charged and the business is not left with disputed balances.
- **Payments.** Wallet funding through Paystack Pay with Transfer, with signed webhooks, idempotent crediting, and amount and expiry checks. A wallet ledger with row-locked debits keeps a record of every money movement.
- **Provider layer.** Live purchases through the DodoData API behind a swappable provider layer (VTPass supported), so the supplier can change with one setting. Only explicit provider failures are refunded; unclear replies stay pending for manual review.
- **Security and admin.** JWT authentication, hashed transaction PIN, OTP-based password reset and rate limiting on sensitive endpoints. Admin tools settle pending transactions, set plan prices, review funding requests and publish announcements.
- **Frontend.** Installable React (Vite) PWA with a guided data-purchase flow, transaction history (Successful, Pending, Failed, Refunded), light and dark themes, and Swagger API docs.
  Stack: Python, Django, DRF, PostgreSQL (Neon), React (Vite), JWT, Paystack, DodoData, Render, Vercel

SMD (Salisu Mahadi Data) – VTU Reseller Platform *Freelance · Paying client*

Backend Developer Live link | GitHub

- **Problem.** A reseller client needed a platform to sell airtime and data on MTN, Airtel, Glo and 9mobile, with customer money handled safely.
- **Built.** End to end, from requirements to deployment: a wallet with deposits and withdrawals, a bcrypt-hashed transaction PIN and UUID-referenced transaction history. Used VTPass in development and switched to DodoData for production.
- **Impact.** Idempotent requests and automatic refunds on VTPass failure codes 012, 014 and 016 meant a failed purchase returned the customer's money instead of leaving it stuck, and a retried request did not charge twice.
- **Trade-off.** A server-rendered, session-based purchase flow with no JavaScript frontend: less interactive, but simpler to secure and faster to deliver.
- **Delivery.** Deployed on Render with PostgreSQL, Gunicorn and WhiteNoise. It served real users while active and is no longer in service.

  Stack: Python, Django, PostgreSQL, VTPass, DodoData, Bootstrap, Render

Student Resource Hub (SRH) *Personal project · 2025 – 2026*

Backend and Frontend Developer Live link | Backend repo

- **Problem.** A friend could not recover a course PDF from five months earlier, and past questions were scattered across WhatsApp groups. SRH is one permanent, searchable library organised by faculty, department and level.
- **Impact.** 100 users within two days of the testing phase. Testers described the same pain unprompted, which confirmed it solves a real problem.
- **Built.** Django REST Framework API and React (Vite) PWA: JWT auth, student, lecturer and admin roles, search, ratings, threaded comments, notifications, download tracking, and password reset by email through Resend.
- **Reliability.** Fixed a production database failure caused by the backend and database sitting in different Render regions. Rebuilt a failed backup into a verified dump and restore routine after about 50 test accounts were lost, and confirmed a full restore end to end.
- **Trade-off.** Anyone can submit, but a department moderator approves each upload, instead of locking uploads behind permissions students dislike. Schema changes stay additive so the live database is never restructured.

  Stack: Python, Django, DRF, PostgreSQL, JWT, React (Vite), Cloudinary, Resend, Render, Vercel

Auction Marketplace API *Technical assessment · 2026*

Backend Developer GitHub

- **Context.** Built in under three days for a volunteer engineering role, and accepted onto the team.
- **Impact.** Four enforced bidding rules keep auctions fair: a bid must exceed the current price, owners cannot bid on their own listing, expired auctions reject bids, and the current price updates atomically.
- **Security.** JWT with token blacklisting on logout, an email-based custom user model with roles, and an object-level permission class (IsOwnerOrAdminOrReadOnly).
- **Quality.** 30+ tests across models, views and permissions, plus Swagger/OpenAPI docs. Auction closing logic lives once on the model and is reused by the API and a management command.

  Stack: Python, Django, DRF, JWT, drf-spectacular

Savewizely – Locked Savings API *Personal project · In progress*

Backend Developer API docs | GitHub

- **Problem.** Everyday money apps make spending too easy. Savewizely lets users lock funds until a chosen date or goal, with early withdrawal allowed for a penalty fee.
- **Built so far.** Django REST Framework backend with a custom user model, wallet, transaction and savings models, account endpoints (register, login, password reset) and a webhook, documented in a Postman collection. Backend first, so the money logic is solid before any UI exists.

  Stack: Python, Django, DRF, PostgreSQL, Postman

EDUCATION

B.Sc. Information Technology, Sule Lamido University, Kafin Hausa *2024 – 2028*

Won 1st place at the Sule Lamido University hackathon, 2026.